

Progetto

Si consideri la seguente estensione del lambda calcolo tipato semplicemente visto a lezione aggiungendo i costrutti if-then-else e le costanti true e false.

```
t ::= x (variable)
    |  $\lambda x:T, t$  (abstraction)
    | t t (application)
    | true (constant true)
    | false (constant false)
    | if t then t else t (conditional)
```

I tipi sono:

$T ::= \text{Bool} \mid T \rightarrow T$

La sostituzione è definita come:

1. $x [x/s] = s$
2. $y [x/s] = y$ if $x \neq y$
3. $(\lambda x:T, t) [x/s] = \lambda x:T, t$
4. $(\lambda y:T, t) [x/s] = \lambda y:T, t[x/s]$ if $x \neq y$
5. $(t_1 t_2) [x/s] = (t_1[x/s]) (t_2[x/s])$
6. **true** $[x/s] = \text{true}$
7. **false** $[x/s] = \text{false}$
8. **(if** t_1 **then** t_2 **else** $t_3)$ $[x/s] =$
 if $t_1 [x/s]$ **then** $t_2 [x/s]$ **else** $t_3 [x/s]$

I valori sono esclusivamente i termini della forma $\lambda x:T, t$ più le costanti true e false.

Sia **value** il predicato sui termini che dice se un termine è un valore o meno.

Chiamiamo **programmi** i termini che non hanno variabili libere.

La semantica **call-by-value** per i programmi è data dalle seguenti regole:

$$\frac{\text{value } v_2}{(\lambda x:T_2, t_1) \ v_2 \mapsto t_1[x/v_2]} \text{ beta}$$

$$\frac{t_1 \mapsto t_1'}{t_1 \ t_2 \mapsto t_1' \ t_2} \text{ app1}$$

$$\frac{\begin{array}{c} \text{value } v_1 \\ t_2 \mapsto t_2' \end{array}}{v_1 \ t_2 \mapsto v_1 \ t_2'} \text{ app2}$$

$$\frac{}{(\text{if true then } t_1 \text{ else } t_2) \mapsto t_1} \text{if true}$$

$$\frac{}{(\text{if false then } t_1 \text{ else } t_2) \mapsto t_2} \text{if false}$$

$$\frac{t_1 \mapsto t_1'}{(\text{if } t_1 \text{ then } t_2 \text{ else } t_3) \mapsto (\text{if } t_1' \text{ then } t_2 \text{ else } t_3)} \text{if}$$

Con $\Rightarrow$ denotiamo la chiusura riflessiva e transitiva di $\rightarrow$

Domanda: limitandoci ai “programmi” la semantica call by value elimina il problema di definire che un termine t sia libero per x in un altro termine v ?

Il type system è così definito (le regole standard più quelle per i booleani e l' if-then-else

$$\begin{array}{c}
 \frac{}{\Gamma, x:T \vdash x : T_1} \text{ base} \\
 \\
 \frac{\Gamma, x: T_2 \vdash t_1 : T_1}{\Gamma \vdash \lambda x:T_2, t_1 : T_2 \rightarrow T_1} \text{ lambda} \\
 \\
 \frac{\Gamma \vdash t_1 : T_2 \rightarrow T_1 \quad \Gamma \vdash t_2 : T_2}{\Gamma \vdash t_1 t_2 : T_1} \text{ app} \\
 \\
 \frac{}{\Gamma \vdash \mathbf{true} : \text{Bool}} \text{ True} \\
 \\
 \frac{}{\Gamma \vdash \mathbf{false} : \text{Bool}} \text{ False} \\
 \\
 \frac{\Gamma \vdash t_1:\text{Bool} \quad \Gamma \vdash t_2:T_1 \quad \Gamma \vdash t_3:T_1}{\Gamma \vdash \mathbf{if } t_1 \mathbf{ then } t_2 \mathbf{ else } t_3:T_1} \text{ ite}
 \end{array}$$

Dopo aver implementato sintassi e semantica si dimostrano i seguenti teoremi in Coq.

- 1) $\neg(\exists S\ T, \vdash \lambda x:S, x\ x : T)$.
- 2) $\vdash \lambda x:\text{Bool} \rightarrow B, \lambda y:\text{Bool} \rightarrow \text{Bool}, \lambda z:\text{Bool}, y\ (x\ z) : T$.
- 3) $(\lambda x:\text{Bool} \rightarrow \text{Bool}, x)\ ((\lambda x:\text{Bool}, \text{if } x \text{ then false else true})\ \text{true}) \Rightarrow \text{false}$
- 4) **Subject Reduction:**
 $\forall t\ t'\ T, (\vdash t : T \wedge t \Rightarrow t') \Rightarrow \vdash t' : T$.